

ShopKart

Full-Stack E-Commerce Web Application

Complete Project Explanation
Key File / Line-by-Line Walkthrough
Interview Questions & Answers

Tech: React (Vite) + Node.js/Express + MySQL + JWT
Also deployable as a static SPA (Vercel / Netlify) with browser fallbacks

Prepared for college project viva / software interview preparation.
Repository structure: shopkart/client (frontend), shopkart/server (backend).

1. Project Overview

ShopKart is a full-stack e-commerce web app built as a college project. Users can browse products, create accounts, manage a cart and wishlist, place Cash on Delivery (COD) or mock UPI orders, and view order history. Admins can manage products and update order statuses.

Important: Online UPI checkout is a mock demo flow for learning purposes. It does not connect to a real payment gateway like Razorpay or Stripe.

1.1 Problem it solves

- Gives a complete shopping flow from browse to order tracking
- Separates user and admin roles with JWT authentication
- Persists cart, wishlist, and orders in MySQL for local/full-stack mode
- Can still run as a static website online (Vercel/Netlify) using browser localStorage + a demo product catalog when no backend is available

1.2 Tech stack

- Frontend: React 19, React Router 7, Vite 8, plain CSS
- Backend: Node.js, Express 4
- Database: MySQL (mysql2 pool)
- Auth: JWT (jsonwebtoken) + password hashing (bcryptjs)
- Deploy: Vercel / Netlify for frontend; Express+MySQL for full API

2. Architecture

2.1 High-level design

ShopKart uses a client-server architecture. The React SPA talks to a REST API. In local development, Vite proxies /api requests to Express on port 5000. Express reads/writes MySQL. Protected routes require a Bearer JWT.

```

Browser (React SPA :5173)
  | fetch('/api/...')
  v
Vite proxy (dev) -----> Express API (:5000)
                          |
                          v
                        MySQL (shopkart DB)
  
```

2.2 Two runtime modes

A) Full-stack local mode (recommended for demos with MySQL)

- Start server: `cd server && npm run dev`
- Start client: `cd client && npm run dev`
- All auth/cart/wishlist/orders persist in MySQL

B) Static production mode (Vercel/Netlify without backend)

- Products come from `client/src/data/demoCatalog.js` (42 items)
- Auth uses `client/src/localAuth.js` (localStorage users + tokens)

- Cart/orders use client/src/localShop.js
- Wishlist uses localStorage via WishlistContext

This dual-mode design is useful in interviews: it shows you understand frontend-only hosting limits and graceful API fallbacks.

3. Features

User features

- Signup / Login / Logout with JWT session restoration
- Profile update (name, email, phone, address, password)
- Browse products with search, category filters, and sort
- Product detail page with quantity, Add to bag, Buy now, Wishlist
- Cart quantity update/remove and checkout
- Wishlist add/remove and Move to cart
- Orders list + order detail with payment method/status
- Light/dark theme toggle (saved in localStorage)

Admin features

- Admin dashboard stats
- Product create / update / delete
- Order status updates: pending, confirmed, shipped, delivered, cancelled

Default seeded admin (local/demo): admin@shopkart.com / admin123

4. Project Structure

```
shopkart/
  client/
    src/
      api.js           Shared fetch helper + fallbacks
      localAuth.js     Browser auth for static hosting
      localShop.js     Browser cart + orders
      data/demoCatalog.js Demo products (42)
      context/         Auth, Cart, Wishlist, Theme
      components/      Navbar, ProductCard, etc.
      pages/           Route screens
      App.jsx          Routes + providers
      main.jsx         React entry point
  server/             Express backend
    server.js          App bootstrap
    config/db.js       MySQL pool
    middleware/auth.js JWT + admin checks
    routes/            auth, products, cart, wishlist, orders, admin
    sql/schema.sql     Database tables
  netlify.toml / vercel.json  Deploy config
  netlify/functions/auth.js  Optional Netlify auth function
```

5. Database Design (MySQL)

Schema file: server/sql/schema.sql

- users: id, name, email, password (hash), phone, address, role, created_at
 - products: id, name, description, price, image_url, category, stock
 - cart_items: id, user_id, product_id, quantity (unique user+product)
 - wishlist: id, user_id, product_id (unique user+product)
 - orders: id, user_id, total_amount, status, payment_method, payment_status, shipping_address, phone, created_at
 - order_items: id, order_id, product_id, quantity, price
- Relationships: users 1-N cart_items/wishlist/orders; products 1-N cart/wishlist/order_items; orders 1-N order_items. This is a classic e-commerce relational model.

6. Authentication Flow (JWT)

- 1) User signs up or logs in with email/password.
- 2) Server hashes passwords with bcrypt (never stores plain text).
- 3) Server signs a JWT containing { id, email, role } with JWT_SECRET, expiry 7 days.
- 4) Client stores token in localStorage key shopkart_token.
- 5) apiRequest attaches Authorization: Bearer <token> on each call.
- 6) authenticate middleware verifies token and sets req.user.
- 7) requireAdmin blocks non-admin users from /api/admin/*.
- 8) On app load, AuthContext calls GET /auth/me; invalid token is cleared.

7. REST API Endpoints

Auth

- POST /api/auth/signup, POST /api/auth/login
- GET /api/auth/me, PUT /api/auth/profile (auth required)

Products

- GET /api/products, GET /api/products/categories, GET /api/products/:id

Cart (auth)

- GET /api/cart, POST /api/cart, PUT /api/cart/:id, DELETE /api/cart/:id

Wishlist (auth)

- GET/POST /api/wishlist, DELETE /api/wishlist/:productId
- POST /api/wishlist/:productId/move-to-cart

Orders (auth)

- POST /api/orders, GET /api/orders, GET /api/orders/:id

Admin (auth + admin role)

- Stats, product CRUD, order status update under /api/admin/*

8. Key Code Walkthrough (Line-by-Line Style)

A true line-by-line dump of every file would be hundreds of pages. Below is a practical interview-style walkthrough of the most important files, explained block by block / line group by line group.

8.1 client/src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode><App /></StrictMode>
);
```

- StrictMode: helps catch unsafe side effects during development
- createRoot(...).render(...): React 18+/19 entry point mounting App
- index.css: global theme variables and component styles

8.2 client/src/App.jsx (routing + providers)

App wraps the tree with ThemeProvider -> AuthProvider -> CartProvider -> WishlistProvider -> BrowserRouter. Then it renders Navbar, <Routes>, Footer.

- Public routes: /, /about, /products, /products/:id, /login, /signup
- Protected routes wrap Cart, Wishlist, Checkout, Profile, Orders
- Admin route uses ProtectedRoute with adminOnly
- ScrollToTop ensures navigation starts at page top (esp. mobile)

8.3 client/src/api.js

Central API helper. Builds URL from VITE_API_URL or '/api'. Adds JSON headers and Bearer token. On production without backend URL, auth/orders use local handlers; product GETs fall back to demoCatalog when response is not JSON.

- apiRequest(path, options): main function used by pages/contexts
- shouldUseLocalAuth(): true in PROD when VITE_API_URL is missing
- This is why the live Vercel site still supports signup/cart/orders

8.4 client/src/context/AuthContext.jsx

- On mount: if shopkart_token exists, call /auth/me and set user
- login(token, user): saves token + sets user state
- logout(): clears token and user
- isAdmin: user?.role === 'admin'

8.5 client/src/context/CartContext.jsx

- Keeps items, total, itemCount in React state
- refreshCart/addToCart/update/remove call API or localShop helpers
- Navbar bag badge reads itemCount from this context

8.6 client/src/context/WishlistContext.jsx

- Tracks wishlist items and a Set of productIds for heart toggles
- toggleWishlist adds or removes; moveToCart adds to cart then removes

8.7 client/src/components/ProductCard.jsx

- Shows image, category, name, rating, price, MRP, discount badge
- Heart button calls toggleWishlist (redirects to login if guest)
- Add to bag button calls addToCart (redirects to login if guest)
- Same card component is reused on Home and Products (consistent UI)

8.8 client/src/components/ProtectedRoute.jsx

- While auth loading: show checking message
- If no user: Navigate to /login with state.from for redirect-back
- If adminOnly and not admin: Navigate away from /admin

8.9 client/src/pages/Checkout.jsx

- Validates shipping address and 10-digit phone
- Payment: COD or mock UPI (Pay now sets upiPaid=true after delay)
- POST /orders with paymentMethod + paymentConfirmed
- On success: navigate to order detail and clear cart state

8.10 server/server.js

```
app.use(cors());
app.use(express.json());
app.use('/api/auth', authRoutes);
app.use('/api/products', productRoutes);
app.use('/api/cart', cartRoutes);
app.use('/api/wishlist', wishlistRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/admin', adminRoutes);
await testConnection(); app.listen(PORT);
```

- cors(): allows React origin to call API
- express.json(): parses JSON request bodies
- testConnection(): fails fast if MySQL credentials are wrong
- EADDRINUSE handler: helpful message if port 5000 is already taken

8.11 server/middleware/auth.js

- Reads Authorization header; must start with 'Bearer '
- jwt.verify(token, JWT_SECRET) decodes payload
- Attaches req.user = { id, email, role }
- requireAdmin returns 403 if role is not admin

8.12 server/routes/auth.js (concept)

- Signup: validate -> check duplicate email -> bcrypt.hash -> INSERT user
- Login: find user by email -> bcrypt.compare -> createToken
- Profile: update fields; optional password change with current password

8.13 server/routes/cart.js (concept)

- All routes use authenticate middleware
- Add to cart validates stock and uses INSERT ... ON DUPLICATE KEY UPDATE
- GET cart joins cart_items with products for name/price/image/line_total

8.14 server/routes/orders.js (concept)

- Uses a DB transaction: lock cart rows, validate stock, insert order
- Inserts order_items, decrements product stock, clears cart
- UPI orders: payment_status=paid, status=confirmed
- COD orders: payment_status=unpaid, status=pending

8.15 Deploy configs

- vercel.json: build client, publish client/dist, SPA rewrite to index.html
- netlify.toml: similar SPA fallback + optional auth function redirect
- Without SPA rewrite, refreshing /products would show Not Found

9. How to Run Locally

```
1) Create MySQL database: CREATE DATABASE shopkart;  
2) cd server -> copy .env.example to .env -> set DB_USER/DB_PASSWORD/JWT_SECRET  
3) npm install && npm run db:setup (or project setup script)  
4) npm run dev # API on http://localhost:5000  
5) cd ../client && npm install && npm run dev  
6) Open http://localhost:5173
```

10. Interview / Viva Questions & Answers

Q: What is ShopKart?

A: A full-stack e-commerce web app where users browse products, manage cart/wishlist, place COD or mock UPI orders, and admins manage products/orders.

Q: Which architecture did you use?

A: Client-server REST architecture: React SPA frontend + Express REST API + MySQL. JWT for authentication between client and server.

Q: Why React and Vite?

A: React provides component-based UI and reusable cards/contexts. Vite gives fast dev server, HMR, and simple production builds for a SPA.

Q: Why Express and MySQL?

A: Express is a lightweight Node framework for REST APIs. MySQL stores relational data (users, products, cart, orders) with constraints and joins.

Q: How does JWT authentication work in your project?

A: After login/signup the server signs a JWT with user id/email/role. Client stores it and sends Bearer token. Middleware verifies signature and expiry before protected routes run.

Q: Why bcrypt?

A: Passwords must never be stored in plain text. bcrypt hashes passwords with salt so even DB leaks do not expose original passwords.

Q: Difference between authentication and authorization here?

A: Authentication = proving identity (login/JWT). Authorization = permission check (requireAdmin allows only role=admin on admin APIs/pages).

Q: How is cart data stored?

A: In full-stack mode, cart_items table keyed by user_id + product_id. In static hosting mode, cart is stored in localStorage via localShop.js.

Q: How do you prevent overselling stock?

A: When adding to cart or placing an order, server checks quantity against product stock. Order placement uses a transaction and decrements stock.

Q: Explain checkout payment options.

A: COD marks payment unpaid and order pending. Mock UPI requires a simulated Pay now confirmation; then payment is marked paid and order confirmed. No real payment gateway is integrated.

Q: What is ProtectedRoute?

A: A React wrapper that redirects unauthenticated users to /login and optionally blocks non-admins from /admin.

Q: Why use Context API?

A: Auth, cart, wishlist, and theme are global UI state. Context avoids prop drilling and lets Navbar badges and pages share the same data.

Q: How does search/filter/sort work?

A: Products page reads URL search params (search, category, sort) and requests filtered products from API or demoCatalog handlers.

Q: What happens if the API is down on Vercel/Netlify?

A: Product GETs fall back to demoCatalog. Auth/cart/wishlist/orders use localStorage implementations so basic shopping still works in demo mode.

Q: What is CORS and why did you enable it?

A: Browsers block cross-origin requests by default. React (5173) and API (5000) are different origins in dev, so Express uses cors() middleware.

Q: Explain a DB transaction in order placement.

A: Begin transaction, validate cart/stock, insert order + items, reduce stock, clear cart, commit. On error rollback so data stays consistent.

Q: What security practices did you follow?

A: Password hashing, JWT secret in env, auth middleware on private routes, input validation, role checks for admin, no secrets committed in code.

Q: What is the difference between PUT and POST in your APIs?

A: POST creates resources (signup, add cart item, place order). PUT updates existing resources (profile, cart quantity, admin product/order status).

Q: How would you improve this project further?

A: Real payment gateway, server-side pagination, image uploads, email invoices, refresh tokens, rate limiting, unit/integration tests, hosted MySQL backend for production API.

Q: Why SPA rewrite is needed on Vercel/Netlify?

A: React Router handles routes in the browser. Without rewrite to index.html, refreshing /products asks the host for a real file and returns Not Found.

Q: What is the role of Vite proxy?

A: In development, requests to /api are forwarded to localhost:5000 so the frontend can call relative URLs without CORS

pain during local coding.

Q: Normalize your database briefly.

A: Users, products, and orders are separate tables. Bridge/line tables (cart_items, wishlist, order_items) avoid repeating product data and keep relations clean (approx. 3NF for core entities).

Q: How do you handle expired JWT?

A: jwt.verify throws; middleware returns 401. AuthContext /auth/me catch clears token and treats user as logged out.

Q: Frontend state vs database state?

A: React state is temporary UI state. MySQL (or localStorage fallback) is persistent source of truth across reloads/devices (DB) or browser (fallback).

11. One-Minute Project Pitch

ShopKart is a full-stack shopping website I built with React, Express, and MySQL. Users can register, browse a catalog, manage cart and wishlist, and place COD or mock UPI orders. Admins can manage products and order status. I used JWT + bcrypt for secure auth, REST APIs for communication, and Context API for shared UI state. For static hosting, I also implemented graceful localStorage fallbacks so signup, cart, and checkout still work without a live backend.

12. Useful Commands Cheat Sheet

```
cd server && npm run dev
cd client && npm run dev
cd client && npm run build
MySQL: CREATE DATABASE shopkart;
Admin demo: admin@shopkart.com / admin123
```